

ESAF Playwright Automation

Playwright suite for ESAF CRM and CBS, with a built-in browser-based **load testing harness** that produces an industry-standard performance report (percentiles, throughput, error rate, Apdex, SLO verdict, time series).

- Functional specs: `tests/CRM`, `tests/CBS`, `tests/AOCO`
- Load specs: `tests/load`
- Full load-testing reference: `LOAD_TESTING.md`

1. Setup

```
npm install
npx playwright install chromium
```

Node 18+ is required (validated on Node 25).

2. Load test quick start

Each virtual user (VU) is a real Chromium session that logs in **once** and then loops its business flow until the duration or iteration cap is reached. Every step is timed as a named transaction.

Always start with a smoke run to validate credentials, data and selectors:

```
npm run load:smoke           # CRM: 2 VUs per flow, 2 iterations each
```

Then run the real profile:

```
LOAD_TEST=1 VUS=10 RAMP_UP_SEC=120 DURATION_SEC=600 npx playwright test tests/load
```

Open the report when it finishes:

```
open load-results/<runId>/index.html
```

`LOAD_TEST=1` is what switches Playwright into load mode: workers scale to the VU count, retries/trace/video/screenshots are disabled so they cannot skew timings, and the aggregated report is generated in the global teardown.

3. Available flows

Flow	Key	Transactions
CRM CASA 360	<code>casa360</code>	Login, search account, open details, Financial Details tab
CRM Customer 360	<code>customer360</code>	Login, search customer, open details, Communication tab, KYC Info tab
CBS Cash Deposit	<code>cashDeposit</code>	Login, open task 1401, enter transaction, denomination details, authorise

Default `FLOWS` is `casa360,customer360`. **`cashDeposit` is never included by default because it posts real financial transactions** — it must be requested explicitly.

4. Commands

```
# CRM - read-only flows
npm run load           # all default flows, configured profile
npm run load:smoke     # 2 VUs per flow, 2 iterations
npm run load:casa360
npm run load:customer360

# CBS Cash Deposit - WRITES DATA, posts and authorises real deposits
npm run load:cashdeposit:smoke  # 1 VU, 1 deposit - always run this first
```

```

npm run load:cashdeposit          # configured profile

# Watch a run in a visible browser
LOAD_TEST=1 HEADLESS=false VUS=1 ITERATIONS=1 npx playwright test tests/load

# Rebuild the report without re-running the test
npm run load:report              # latest run
LOAD_RUN_ID=20260818-143000 npm run load:report

# Functional (non-load) runs – load specs are excluded automatically
npm test
npm run test:headed

```

Fully specified example

```

LOAD_TEST=1 \
FLOWS=casa360,customer360 \
BASE_URL=https://dr-crm.esafbank.org \
TEST_ENV=dr \
VUS=25 RAMP_UP_SEC=300 DURATION_SEC=1800 \
THINK_TIME_MIN_MS=1000 THINK_TIME_MAX_MS=3000 \
SLO_P95_MS=5000 SLO_P99_MS=8000 SLO_ERROR_RATE_PCT=2 APDEX_T_MS=3000 \
npx playwright test tests/load

```

5. Test data

Data lives in CSVs. Rows are **grouped by username/password**: a VU is assigned one user, logs in once as that user, and cycles only through that user's records — so the session and the data always belong to the same user.

Flow	File	Columns
casa360	fixtures/casa360.csv	username,password,casaNumber
customer360	fixtures/customer360.csv	username,password,customerId
cashDeposit	fixtures/cbs_cashDeposit.csv	username,password,accountNo,amount,narrative,noteDenomination,authorizer

Lines starting with **#** are comments. Override paths with **CASA360_DATA**, **CUSTOMER360_DATA**, **CASHDEPOSIT_DATA**.

Before any sizeable run, add one application user per VU. VUs are assigned users round-robin, so 25 VUs against 2 users puts ~13 concurrent sessions on each user; you would be measuring server-side session/lock contention rather than capacity. This is critical for **cashDeposit**, where CBS teller tills and maker/checker queues are stateful per user.

For **cashDeposit**, **amount** must be exactly divisible by **noteDenomination** — the note count is derived as **amount / noteDenomination** and validated before the run starts.

6. Key environment variables

Variable	Default	Meaning
LOAD_TEST	–	Must be 1 to enable load mode
FLOWS	casa360,customer360	Flows to run: casa360 , customer360 , cashDeposit
VUS	10	Virtual users per flow
RAMP_UP_SEC	60	VUs start linearly across this window
DURATION_SEC	600	Steady-state duration per VU
ITERATIONS	0 (duration driven)	Hard cap on iterations per VU
THINK_TIME_MIN_MS / THINK_TIME_MAX_MS	1000 / 3000	Pause between iterations
BASE_URL	https://dr-crm.esafbank.org	CRM under test
CBS_BASE_URL	https://dr-cbs- uat.esafbank.org:8443/DashboardNG.html	CBS under test

Variable	Default	Meaning
TEST_ENV	dr	Environment label stamped on the report
HEADLESS	true	false to watch the browsers
SLO_P95_MS / SLO_P99_MS / SLO_ERROR_RATE_PCT	5000 / 8000 / 2	Report pass/fail thresholds
APDEX_T_MS	3000	Apdex satisfied threshold
LOAD_RUN_ID	timestamp	Run id / output folder name

Full list in `LOAD_TESTING.md`.

7. Results

Everything is written to `load-results/<runId>/`:

Artefact	Use
<code>index.html</code>	Start here — KPIs, SLO verdict, charts, all tables
<code>summary.json</code>	Machine-readable report for CI gates and trending
<code>transactions.csv</code>	Per transaction: count, pass/fail, min/avg/median/p90/p95/p99/max, std dev, TPS, Apdex
<code>flows.csv</code>	Same statistics per flow
<code>timeseries.csv</code>	Active VUs, throughput, errors and p95 over time
<code>samples.csv</code>	Raw log, one row per transaction execution
<code>errors.csv</code>	Grouped failures with counts and first/last seen
<code>http-metrics.csv</code>	Requests, 4xx, 5xx, failed requests, MB received, slowest request
<code>browser-metrics.csv</code>	TTFB, DOM interactive, DCL, load, FCP (avg and p95)
<code>screenshots/</code>	One screenshot per failed iteration

How to read it

1. **Error rate first** — high error rates invalidate the response times.
2. **p95 / p99** — the headline numbers; averages hide the tail.
3. **Throughput vs active VUs** — when throughput flattens while VUs keep rising, the system has saturated. That knee is the capacity limit.
4. **Re-login count** — non-zero means the server dropped sessions under load.
5. **HTTP 5xx / failed requests** — server-side saturation signals.

Client-side numbers alone are not a diagnosis: capture application server, DB and infrastructure metrics over the same window and attach them to the run folder.

8. Capacity and safety notes

- **One VU = one Chromium instance** (~250–400 MB RAM, ~0.3 vCPU). Beyond roughly 20–25 VUs on a laptop the client is the bottleneck and you are measuring your own machine. Distribute across agents for higher concurrency.
- **cashDeposit writes data.** Every iteration posts and authorises a real deposit; a 5 VU / 10 minute run creates hundreds of postings and moves till and account balances. Run only against UAT/DR with sign-off and an agreed clean-up plan.
- `load-results/` is git-ignored. Archive the folder if you need to keep a run.

9. Troubleshooting

Symptom	Cause / fix
FLows must contain at least one of: ...	Typo in <code>FLows</code> ; keys are case-sensitive (<code>cashDeposit</code>)
... must contain username and password columns for every row	A data CSV row is missing credentials
amount ... is not divisible by noteDenomination ...	Fix <code>amount/noteDenomination</code> in <code>fixtures/cbs_cashDeposit.csv</code>
[load-report] No samples found	Run ended before any transaction completed, or wrong <code>LOAD_RUN_ID</code>

Symptom	Cause / fix
Every VU fails at 00 Login	Wrong credentials, wrong BASE_URL/CBS_BASE_URL, or the environment is down
Timings look implausibly slow	Client saturation — reduce VUS or check machine CPU/RAM during the run